HelixTranslate

HelixTranslate

Tagline: Verified-model book translation — honest by design, never a silent fallback.

Summary: HelixTranslate is a high-performance, Go-based ebook translation platform that translates books between 100+ languages using verified LLM providers, with real-time WebSocket monitoring and a strict no-silent-fallback routing policy that fails loudly rather than degrade quietly.

Short description: Go-based universal ebook translation toolkit. Translates FB2, EPUB, TXT, HTML, PDF, and DOCX across 100+ languages using the strongest verified LLM (via the LLMsVerifier bridge), with REST/HTTP-3 and gRPC APIs, distributed processing, and a real-time WebSocket monitoring dashboard. (~40 words)

Long description: HelixTranslate is an enterprise-grade, Go-based system for translating whole books between languages using LLM providers. It parses and regenerates multiple ebook formats (FB2, EPUB, TXT, HTML, PDF, DOCX), supports 100+ languages with auto-detection, and offers both CLI tools and API servers (REST with HTTP/3, gRPC, and a WebSocket event stream). Its defining characteristic is *how it chooses a model*: rather than hardcoding providers, HelixTranslate delegates all model authority to the LLMsVerifier bridge (`pkg/bridge`), which selects the strongest *verified* API model and a deterministic, score-ordered fallback chain. Model eligibility follows a weighted score (responsiveness, code, feature richness, reliability). Crucially, the system enforces a “no silent fallback” mandate in code: if no provider API key is present, or an operator explicitly requests an unavailable provider, the pipeline returns an honest hard error instead of quietly switching providers or dropping to a local runtime — a rule verified by a dedicated pre-build gate and mutation test. Local runtimes (Ollama, llama.cpp) were deliberately removed from the default path. Around the translation core sits a real-time WebSocket monitoring subsystem: a translation CLI emits typed events to a monitoring server, which drives a live web dashboard, and remote SSH workers enable

distributed translation. Additional capabilities include multi-pass polishing, a preparation-phase quality analysis, translation caching, and vision-driven QA. The platform is governed by an anti-bluff engineering constitution: tests must prove real user-visible outcomes, backed by mandatory mutation testing.

Why we built it: To translate long-form books reliably and *honestly* — never shipping a “degraded-but-present” translation. The design premise is that a missing or unverifiable translation must be a loud, hard error, and that model selection must always resolve to a genuinely verified provider rather than a hardcoded guess or a silent local fallback. (Note: the repo has no single canonical “mission” prose paragraph; this rationale is synthesized from the constitution and routing code — verify wording before quoting.)

Why it’s a game-changer: Most LLM translation pipelines fail silently — they fall back to a weaker model, a local runtime, or emit partial output while tests still pass green. HelixTranslate makes silent degradation structurally impossible: model choice is verification-gated, the fallback chain is deterministic and transparent, and “no key / no verified model” is an honest error, not a shrug. That turns “did the translation actually happen on a capable model?” from a hope into a guarantee.

What’s innovative: - **Verification-gated model routing** via the LLMsVerifier bridge — the strongest *verified* model is auto-selected; operators don’t have to pick a provider. - **No-silent-fallback guarantee enforced in code** — four explicit routing arms (mock / explicit-verifier / explicit-provider / bridge-default), each of which hard-errors rather than silently switching; deliberate removal of local runtimes from the default path. - **Mechanical enforcement** — a `CM-NO-LOCAL-RUNTIME` pre-build gate plus a paired mutation test asserts no local-runtime client is constructed on the default path. - **Deterministic, score-ordered fallback chain** — provider-to-provider failover among verified models is allowed and transparent (distinct from a forbidden silent fallback). - **Real-time WebSocket monitoring** — typed translation events streamed to a live dashboard; distributed SSH workers. - **Anti-bluff testing regime** — mutation testing, negative assertions, real-system runs, and vision-driven QA so “tests pass” cannot mask “feature doesn’t work.”

Biggest technical challenges + how solved: - **Guaranteeing an honest translation pipeline (no silent degradation).** Solved by centralizing all model authority in the LLMsVerifier bridge, encoding four explicit routing branches that each fail loud, removing local-runtime fallbacks, and enforcing the rule with a build gate + mutation test. - **“Green tests, broken features.”** The constitution explicitly names the problem — tests and challenges pass while features don’t actually work — and solves it with the Anti-Bluff Testing regime: concrete user-visible

assertions, real systems (mocks only in unit tests), mandatory mutation testing (deliberately break the feature → the test must fail), and vision-verified QA. (No dedicated “hardest challenges” doc exists; this is the strongest sourced example.) - **Long-form, multi-format quality.** Multi-pass polishing, a preparation-phase analysis, and translation caching address consistency and cost over book-length inputs.

Tech stack (why + how): - **Go** — high-concurrency backend for parsing, translating, and streaming; module `digital.vasic.translator`. (Version stated inconsistently across `VERSION`/Makefile/`AGENTS.md` — treat as unsettled.) - **Gin** — HTTP/REST API server. - **QUIC / HTTP/3 (quic-go)** — low-latency, modern transport for the REST API. - **gRPC + Protocol Buffers** — high-performance service interface alongside REST. - **Gorilla WebSocket** — real-time translation-event stream to the monitoring dashboard. - **PostgreSQL, SQLite, Redis** — relational + embedded storage + cache; SQLite also backs the bridge’s verified-models store (`data/verified_models.db`). - **unidoc/unioffice + unipdf** — DOCX and PDF document processing for multi-format ebooks. - **Cobra** — CLI framework for the `unified-translator` and companion tools. - **golang-jwt (JWT HS256)** — API authentication; plus per-IP token-bucket rate limiting and TLS/QUIC security. - **LLMsVerifier bridge (`pkg/bridge`)** — sources the strongest verified model + deterministic fallback chain; the enforcement point for no-silent-fallback. - **Testify** — Go test suite, including the dedicated `provider_routing_test.go` and mutation gates. - **Docker / Podman (rootless) + Compose** — containerized, distributed deployment (`docker-compose.distributed.yml`).

Public links: - Product repo, PUBLIC: `github.com/HelixDevelopment/HelixTranslate` - Governance reference, PUBLIC: `github.com/HelixDevelopment/HelixConstitution` - **PRIVATE — do NOT publish as links:** SSH origin remotes (`git@github.com:HelixDevelopment/HelixTranslate.git`, `git@github.com:milos85vasic/Translator.git`). Dashboard/monitor endpoints are localhost-only (e.g. `http://localhost:8090/monitor`) — not public. License (README claims MIT) not confirmed against a LICENSE file — verify before stating.

Suggested diagrams/illustrations (OpenDesign): 1. **Provider-routing / no-silent-fallback flow** (signature diagram) — decision tree: request → is provider explicit? → is a verified model available? → strongest-verified selection with deterministic fallback chain; every “no” branch terminating in a red “honest hard error,” with local runtimes shown crossed out. This is the product’s headline concept. 2. **Verification-gated model selection** — LLMsVerifier scores models (responsiveness / code / feature-richness / reliability weights) → only verified, positively-scored models enter the eligible pool → HelixTranslate picks the top one. 3. **Real-time monitoring pipeline** — Translation CLI →

typed WebSocket events → Monitoring Server → live Web Dashboard, with remote SSH workers feeding distributed translation. 4. **Multi-format ebook flow** — FB2/EPUB/PDF/DOCX/HTML/TXT in → parse → translate (verified model) → multi-pass polish → regenerate target format.

Site relevance: Both. Product page on **vasic.digital** (lead with the no-silent-fallback / verified-routing story); portfolio entry on **milosvasic.ru**.

Priority tier: Helix-primary (LLM-infrastructure cluster). Ranks within the Helix platform family, after HelixTrack.

Source provenance: Local repo `/Volumes/T7/Projects/helix_translate` — `README.md`, `CONSTITUTION.md`, `AGENTS.md`, `Documentation/ARCHITECTURE.md`, `cmd/unified-translator/main.go` (routing switch + error strings), `pkg/bridge/bridge.go`, `go.mod`, `VERSION`, `pkg/bridge/provider_routing_test.go`. No-silent-fallback also grounded in `constitution/Constitution.md` (§11.4.69 and the "Silent fallback" clause). Cross-checked against harvested `_analysis/github-helix-others.md`. Caution flags: root README describes the *monitoring* subsystem, not the book translator (product framing lives in AGENTS.md / ARCHITECTURE.md); version and Go-builder versions are inconsistent across files; ARCHITECTURE.md still lists removed Ollama/local engines (stale); WebSocket performance numbers are stated targets, not verified; no canonical "why we built it"/"hardest challenges" prose exists. "Helix-Flow" appears only as an org name in governance boilerplate — not a HelixTranslate component.